Zero Knowledge Proofs

Om Joglekar

Contents

1 Introduction

Most of the material here has been adapted from [2] and also [1]. These notes are prepared from a pure mathematicians point of view. The goal is to give a rigorous mathematical treatment of the subject. We will not be concerned with the practical aspects of implementing ZKPs, but rather with the theoretical foundations and constructions. There is a combinatorial perspective lurking in the background of all the constructions and we will try to highlight that as much as possible.

1.1 First Examples of ZKP's

A zero-knowledge proof is essentially a protocol between a prover and a verifier, where the prover aims to convince the verifier of the truth of a statement without revealing any additional information. In simpler terms, the prover wants to demonstrate that they know a secret (or that a certain condition holds) without actually disclosing the secret itself.

We take a look at several examples to familiarise ourselves with this concept. The first one is the famous Where's Waldo puzzle, where the prover wants to convince the verifier that they know where Waldo is in a crowded scene without revealing his location.

Consider the following Where's Waldo puzzle (Waldo has been shown for reference):

Suppose you are struggling to find Waldo in the image while I have already found him. I want to convince you that I know where Waldo is. However, if I point him out to you directly, you will have learnt where he is and that is a spoiler! You want to be convinced that I know where Waldo is without me revealing his location to you. How can I do this?

One way to do this is to use a big piece of cardboard with a small hole in it. I can place the cardboard over the image and align the hole with Waldo's location. Then, I can show you the cardboard with the hole, allowing you to see only Waldo through it. This way, you are convinced that I know where Waldo is, but you don't learn anything else about his location or the surrounding area.

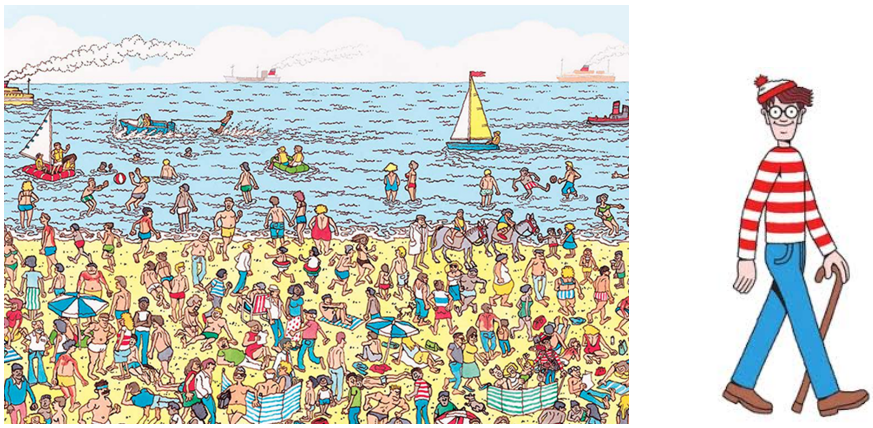

Figure 1: Can you find Waldo? (Target shown on the right)

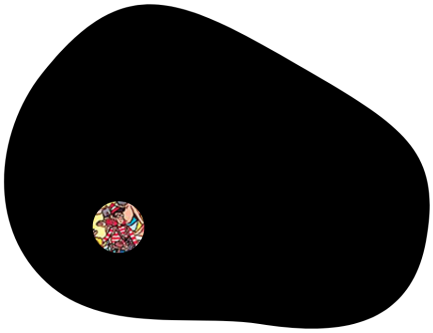

Figure 2: ZKP Solution (sort of) to the Waldo puzzle

In the above figure, I have shown you Waldo in the puzzle but the puzzle itself could have been oriented in infinitely many ways behind the black cardboard.

Another popular example of a ZKP is that of a colour blind man. Suppose there is a colour blind man who has never known that there are two different colours, say, red and blue. You meet him one day and show him two balls, one red and one blue. He laughs and tells you that the two have the same colour. Now you know that they are not the same colour since your vision is normal. You want to convince him that you can distinguish between the two balls but you do not want to tell him at the end which is which.

You simply hand the balls over to him and ask him to hide them behind his back. Then at his own discretion he chooses to switch the balls or not. He then brings them in front of you and since you have a normal vision, you tell him if he has switched them or not. But since this was done once, he can simply retort that your guess whether it was switched or not was lucky. To fix this, you repeat the game multiple times. If the colour blind man insists on rigour, he will of course never be convinced fully unless you play an infinite number of times and he can keep calling you lucky every time you 'guess'. But he should be able to believe you with high probability!

We give a third example of a ZKP, which is the famous Ali Baba's cave puzzle. There is a cave with two paths, A and B, that lead to a locked door. The door can only be opened with a secret password. The prover claims to know the password and wants to convince the verifier without revealing it. The verifier stands outside the cave while the prover goes inside and chooses either path A or B. The verifier then randomly chooses a path and asks the prover to return through that path. If the prover knows the password, they can always comply with the verifier's request. However, if the prover does not know the password, they can only comply with the verifier's request if they happen to choose the correct path. By repeating this process

multiple times, the verifier can be convinced with high probability that the prover knows the password without ever learning what it is.

A simpler version of the above would be the case when you have a circular corridor and there is locked door somewhere which does not allow you to complete the full 360 degree walk around the corridor. You want to convince your friend that you know the password without opening the door in front of him. You stand at a point where the door cannot be seen and then tell your friend to wait there. You then walk around one side, open the door and come out the other. Without knowing how to open the door, this would have been impossible and your friend is convinced.

We now give two mathematical examples of ZKPs. The first one is the problem of determining if a graph admits a Hamiltonian cycle. Suppose you and I are staring at a massive graph. The goal is to find a Hamiltonian cycle (cycle that visits every vertex exactly once). You start to give up and then I tell you that I have found it. I could tell you the order of the vertices and you could nod and accept my solution. But then I would have sucked the fun out of it for you. Instead I give you a ZKP that I found the cycle.

To achieve this, I first create a secret, randomly permuted version of the graph and construct its adjacency matrix, keeping all the entries hidden from you. Essentially I relabel the n vertices in any one of the $n!$ ways uniformly at random. I then present you with a challenge where you can ask for one of two things, but never both. First, you can ask me to reveal the full permutation to verify that this hidden matrix is indeed isomorphic to our original graph. Alternatively, you can ask me to reveal only the specific sequence of ones in the adjacency matrix that corresponds to the Hamiltonian cycle. These would appear uniquely in a given row or a given column.

If you were allowed to ask for both simultaneously, you could easily map the revealed cycle back onto the original graph, which would instantly expose the secret. By repeating this process multiple times with a new random permutation each round, your confidence grows exponentially, yet you gain zero knowledge about the actual solution.

You might still wonder how this truly convinces you if I only answer half of the challenge each time. The security of this protocol relies on the compounding probability of deception across multiple rounds. If I were lying and did not actually know the Hamiltonian cycle, I could only construct the hidden matrix to satisfy one of your two potential requests—either by making it a valid permutation of the original graph, or by drawing a fake cycle in an unrelated graph. Thus, a scammer has at most a 50% chance of guessing your challenge correctly in a single round. By repeating this process over and over, we can increase the confidence just like the colour blind ZKP.

In general, the problem of finding a Hamiltonian cycle is NP-complete and hence the above ZKP can be used to give a ZKP for any problem in NP. This is because any problem in NP can be reduced to the Hamiltonian cycle problem.

An equivalent favourite example would be that of a 3-colouring problem of graphs. This is of course an NP problem and hence we can use the above ZKP. But there is also the following direct ZKP. The prover first randomly permutes the colours and then colours the graph with the permuted colours. The verifier then asks the prover to reveal the colouring of two adjacent vertices.

1.2 Proof Systems

We now discuss some fundamentals of mathematical proofs. Informally, what we mean by a proof is anything that convinces someone that a statement is true, and a "proof system" is any procedure that decides what is and is not a convincing proof.

There are four obvious properties we expect of a proof system.

(i) **Completeness**: If the statement is true, then there should be a proof that convinces the verifier.

(ii) **Soundness**: If the statement is false, then no proof should be able to convince the verifier.

(iii) **Verifying Efficiency**: Simple statements should have short convincing proofs that can be verified quickly.

(iv) **Proving Efficiency**: Simple statements should have short convincing proofs that can be found quickly.

Now traditionally a proof is a static object. The prover has done his job and has disappeared. The verifier and the piece of proof that the prover left for the verifier are the only two parties left. The verifier can read the proof and check if it is correct. However, in an interactive proof system, the prover and the verifier can have a conversation. The prover can send messages to the verifier and the verifier can ask questions back to the prover. This allows for a much richer class of proofs and can potentially allow for more efficient proofs.

In cryptographic contexts, such interactive proofs prove to be very useful when they have the additional aspect of being zero-knowledge. This means that the verifier learns nothing from the proof other than the fact that the statement is true. This is a very strong property and is what makes zero-knowledge proofs so powerful.

The main goal of this set of notes is to describe a variety of approaches to constructing zero-knowledge succinct non-interactive arguments of knowledge, or zk-SNARKs for short. "Succinct" means that the proofs are short. "Non-interactive" means that the proof is static, consisting of a single message from the prover. "Of Knowledge" roughly means that the protocol establishes not only that a statement is true, but also that the prover knows a "witness" to the veracity of the statement. Let us elaborate a little more on these aspects of zk-SNARKS.

To understand zk-SNARKs, we must define the three properties that complete the acronym: succinctness, non-interactivity, and being an argument of knowledge.

(i) **Succinctness:** The proof size is very small (often just a few hundred bytes), and verification takes only a few milliseconds, regardless of how computationally heavy the underlying statement is.

(ii) **Non-interactivity:** The prover and verifier do not need to engage in a back-and-forth dialogue. The prover generates a single, static proof string that anyone can verify independently at any time.

(iii) **Argument of Knowledge:** This guarantees that the prover actually possesses a valid 'witness' (the secret data or solution), rather than just proving that a valid witness exists abstractly somewhere in the universe.

At first glance, this definition introduces two major contradictions.

First, how can a protocol simultaneously be 'zero-knowledge' (revealing nothing) and an 'argument of knowledge' (proving possession of a secret)? If the verifier learns that I possess the witness, haven't they learned something? The resolution lies in the mathematical formulation of these properties. Zero-knowledge is a guarantee for the actual verifier, ensuring they learn zero bits of information about the witness's structure, value, or whereabouts. The 'of knowledge' property, however, is a guarantee about the prover's capability. It is defined via a hypothetical thought experiment: if an external observer (an extractor) had total control over the prover's execution environment, they could extract the witness. Therefore, the real-world verifier gains no knowledge about the secret itself, but receives a mathematical guarantee that the prover could not have fabricated the proof without actually holding that secret.

Second, if the very foundation of a zero-knowledge proof relies on interactive challenges (like the color-blind man or the Hamiltonian graph permutations), how can a zk-SNARK be 'non-interactive'? Isn't a non-interactive interactive proof a contradiction in terms?

Historically, zero-knowledge proofs did require interaction to prevent the prover from cheating. In a non-interactive system like a zk-SNARK, this interaction is simulated cryptographically. Instead of a live verifier tossing random coins to challenge the prover, the prover uses a cryptographic mechanism—such as a Common Reference String (CRS) set up beforehand, or a cryptographic hash function (via the Fiat-Shamir heuristic)—to generate the challenges themselves. Because the prover cannot predict or manipulate the output of these cryptographic primitives, they are forced to answer the challenges honestly, exactly as if a live verifier were questioning them. This allows the protocol to compress a multi-round conversation into a single, static proof that can be verified asynchronously by anyone.

Currently there are various approaches to developing zk-SNARKS. Some of them which we will study will include IP's, MIP's, PCP's, IOP's and linear PCP's.

The reader must, at this point, note that most of our proofs will be probabilistic and we will compromise on

the soundness of proofs. However, the proof will be sound with high probability.

1.3 Interactive Proof Systems: A Formal Perspective

To understand interactive proofs, it is helpful to look at a practical scenario: a user who delegates data storage and processing to an external server, such as a cloud provider. The user uploads their massive dataset to the server but keeps a tiny, secret summary of that data for themselves. Later, when the user asks the server to compute a function f on that data, the server returns a claimed result. Rather than blindly trusting that the server computed the result honestly, the user can run an interactive proof protocol to formally verify the correctness of the output.

This verification happens through an interrogation phase. The user issues a sequence of unpredictable challenges, and the server must provide matching responses. At the end of this exchange, the user decides whether to accept or reject the answer. The system relies entirely on two core properties:

(i) **Completeness:** If the server runs the program honestly and follows the protocol, the user will always be convinced to accept the answer.

(ii) **Soundness:** If the server returns an incorrect output, the user will detect the lie and reject the answer with high probability, no matter how hard the server tries to trick them.

The magic of an interactive proof lies in the element of surprise. Because a cheating server cannot predict what the user's next challenge will be, it cannot pre-fabricate fake answers, allowing the verifier to easily catch them in a contradiction.

This interaction also highlights a fundamental difference from traditional static proofs: interactive proofs are non-transferable. A static mathematical proof can be copied and handed to a third party, and it remains perfectly convincing. However, a recorded transcript of an interactive proof will not convince anyone else. To an outside observer, the transcript looks exactly like a staged script where the prover and verifier secretly colluded beforehand to make the answers look correct. Soundness only holds if the challenges are generated truly dynamically and blindly in real-time. To make it more explicit, let us refer back to the colour blind example. If the trial was repeated multiple times and the whole video was recorded and showed to a third person, he may say that both of them were colour blind and they had decided the order of the questions and answers beforehand.

1.4 Multi-Prover and Probabilistically Checkable Proofs

While standard interactive proofs involve a single prover, we can expand this setup to Multi-Prover Interactive Proofs (MIPs). An MIP works like a standard interactive proof, but with multiple provers who are completely isolated from one another. A helpful analogy is placing criminal suspects in separate interrogation rooms to see if they can keep their stories straight. Because the provers cannot coordinate or share the verifier's challenges with each other, this isolation allows them to convince the verifier of much more complex statements than if they were allowed to talk.

Another powerful framework is the Probabilistically Checkable Proof (PCP). Unlike an interactive proof, a PCP is a static object, just like a traditional mathematical proof. However, instead of reading the whole thing, the verifier only inspects a few randomly chosen characters from it. This is like a lazy journal referee who wants to check a paper's correctness without reading every line. The famous PCP theorem states that any traditional proof can be rewritten in this format, allowing a reviewer to gain massive confidence in its validity by looking at just a few words.

Philosophically, MIPs and PCPs are groundbreaking, but they have major practical limitations in real-world cryptography due to their strict assumptions:

(i) **MIP Limitations:** Keeping provers isolated is incredibly difficult. In most real-world scenarios, there is either only a single prover, or there is no realistic way to completely block communication between multiple provers.

(ii) **PCP Limitations:** Even though the verifier only reads a tiny piece of the proof, a direct implementation requires the prover to transmit the entire massive proof over the network first. This high transmission cost destroys the benefit of the verifier reading less of it.

Fortunately, we can bypass these real-world limitations by combining MIPs and PCPs with cryptography. This combination allows us to build advanced 'argument systems' where the prover does not have to transmit the entire proof. Ultimately, IPs, MIPs, and PCPs can all be unified under a single framework called *polynomial IOPs* (Interactive Oracle Proofs). By applying a cryptographic primitive known as a polynomial commitment scheme, these systems can be transformed into highly efficient arguments with incredibly short proofs.

References

[1] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Second. Chapman & Hall/CRC, Nov. 2014, p. 603. ISBN: 978-1-4665-7026-9.

[2] Justin Thaler. 'Proofs, Arguments, and Zero-Knowledge'. In: *Foundations and Trends in Privacy and Security* 4.2–4 (2022), pp. 117–660. DOI: 10.1561/3300000030.